

Backtracking

Algorithm Design → Search

Discrete Maths for Engineers

Leandro Rodrigues

March 20, 2026

Contents

1	Series Overview	3
1.1	What Backtracking Solves	3
1.2	Parts	3
2	Problem Model & Search Space	4
2.1	Roadmap	4
2.2	Why Start With the Model?	4
2.3	Candidate Solutions	4
2.4	The Search Space	4
2.5	Constraints	5
2.6	Feasible vs. Complete Solutions	5
2.7	The Objective of Backtracking	5
2.8	Why This Matters	5
3	The Core Idea (Explore + Undo)	6
3.1	Roadmap	6
3.2	Incremental Construction	6
3.3	Explore	6
3.4	Test Early	6
3.5	Dead Ends	6
3.6	Undo	7
3.7	Why Recursion Fits Naturally	7
3.8	The Full Intuition	7
4	Generic Backtracking Algorithm	8
4.1	Roadmap	8
4.2	State Representation	8
4.3	Generating Choices	8
4.4	Feasibility Test	8
4.5	Detecting a Complete Solution	9
4.6	Generic Backtracking Template	9
4.7	Structure of the Search Tree	9
4.8	Why This Template Is Important	9
5	Search Tree & Recursion Structure	11
5.1	Roadmap	11
5.2	The Search Tree	11
5.3	Depth and Decision Levels	11
5.4	Recursion as Tree Traversal	11

5.5	The Call Stack	11
5.6	Why Backtracking Uses DFS	12
5.7	Visual Intuition	12
5.8	Why This Matters	12
6	Pruning	13
6.1	Roadmap	13
6.2	Why Pruning Matters	13
6.3	Constraint-Based Pruning	13
6.4	Early vs Late Pruning	13
6.5	Bounding (Optimization Problems)	13
6.6	Impact on Complexity	14
6.7	Intuition	14
6.8	Summary	14
7	Classic Problems	15
7.1	Roadmap	15
7.2	Permutation Generation	15
7.3	Subset Generation	15
7.4	N-Queens	15
7.5	Sudoku	16
7.6	Graph Coloring	16
7.7	Hamiltonian Path	16
7.8	The Common Structure	16
8	Complexity	18
8.1	Roadmap	18
8.2	Complexity Comes from the Search Tree	18
8.3	Branching Factor	18
8.4	Depth of the Tree	18
8.5	Worst-Case Time Complexity	18
8.6	Examples	19
8.7	Space Complexity	19
8.8	The Effect of Pruning	19
8.9	Main Takeaway	19
9	Backtracking vs DFS vs Dynamic Programming	20
9.1	Roadmap	20
9.2	Backtracking vs DFS	20
9.3	DFS as a Traversal Mechanism	20
9.4	Backtracking as Constrained Search	20
9.5	Backtracking vs Dynamic Programming	21
9.6	Overlapping Subproblems	21
9.7	When DP Replaces Backtracking	21
9.8	Practical Comparison	21
9.9	How to Choose	21
9.10	Series Conclusion	22

1 Series Overview

1.1 What Backtracking Solves

Backtracking is a general technique for exploring a combinatorial search space by incrementally building candidate solutions and abandoning partial states that cannot lead to a valid final answer.

Conceptually, backtracking performs a depth-first exploration of a search tree where each node represents a partial solution, each edge represents a choice, and failed branches are undone so the algorithm can try another possibility.

The presentation follows the classical treatment used in algorithm design and combinatorial search, connecting recursive exploration, pruning, and search trees.

1.2 Parts

- Part 1 — Problem Model & Search Space
- Part 2 — The Core Idea (Explore + Undo)
- Part 3 — Generic Backtracking Algorithm
- Part 4 — Search Tree & Recursion Structure
- Part 5 — Pruning
- Part 6 — Classic Problems
- Part 7 — Complexity
- Part 8 — Backtracking vs DFS vs Dynamic Programming

2 Problem Model & Search Space

2.1 Roadmap

- What kind of problems backtracking solves
- How candidate solutions are modeled
- Why the search space forms a tree
- The role of constraints in pruning
- The precise objective of the method

Backtracking is often introduced informally as a method that tries possibilities and undoes decisions when they fail.

That intuition is useful, but to understand the technique rigorously, we must first define the problem model and the search space it explores.

2.2 Why Start With the Model?

Backtracking is not tied to a single concrete problem. It is a general strategy for exploring a structured set of possibilities under constraints.

Therefore, before discussing recursion or implementation details, we must answer:

- What is a candidate solution?
- How is a partial solution represented?
- When can we stop exploring a branch?

Main idea: backtracking is a method for exploring a constrained search space.

2.3 Candidate Solutions

A candidate solution is usually built incrementally as a sequence of decisions:

$$x = (x_1, x_2, \dots, x_k)$$

Each variable x_i represents a choice made at step i , typically selected from a domain of feasible values.

A complete assignment may represent a full solution, while a prefix $(x_1, \dots, x_i)$ represents a partial solution.

2.4 The Search Space

The collection of all candidate solutions defines the search space.

This space is naturally modeled as a tree:

- The root is the empty solution
- Each level corresponds to one additional decision
- Each edge represents a choice
- Each node represents a partial assignment

In abstract terms, a node may be written as $(x_1, \dots, x_i)$.

2.5 Constraints

Not every partial solution is worth exploring.

A backtracking problem is defined together with constraints that determine whether a partial assignment can still be extended into a valid full solution.

$$C(x_1, x_2, \dots, x_i)$$

If the current partial assignment violates the problem structure, the algorithm stops exploring that branch.

Core pruning principle: reject impossible branches as early as possible.

2.6 Feasible vs. Complete Solutions

It is useful to distinguish two notions:

- Feasible partial solution — a partial assignment that does not violate constraints
- Complete solution — a full assignment that satisfies all problem requirements

Backtracking spends most of its time exploring feasible partial solutions and testing whether they can be extended.

Feasibility is a local condition, while completeness is a global one.

2.7 The Objective of Backtracking

Depending on the problem, the objective may be:

- Find one valid solution
- Find all valid solutions
- Count how many valid solutions exist
- Find an optimal solution under additional criteria

In every case, the method systematically explores the search tree while pruning branches that cannot succeed.

2.8 Why This Matters

This model appears in many classical problems:

- N-Queens
- Sudoku
- Permutation generation
- Subset generation
- Graph coloring
- Hamiltonian path search

In all of them, the central idea is the same: build solutions incrementally, test constraints early, and backtrack whenever a branch becomes impossible.

3 The Core Idea (Explore + Undo)

3.1 Roadmap

- Why backtracking is an incremental exploration method
- How the algorithm moves forward through choices
- Why invalid branches are abandoned early
- What it means to undo a choice
- How recursion matches the search tree structure

Once the search space is modeled as a tree of partial solutions, the central question becomes: how do we move through that tree efficiently?

The essential idea of backtracking is simple: explore one choice, continue recursively, and undo it if necessary.

3.2 Incremental Construction

Backtracking does not generate full solutions all at once. It builds them step by step.

At each stage, the algorithm extends a partial solution by making one additional decision.

$$(x_1, \dots, x_i) \rightarrow (x_1, \dots, x_i, x_{i+1})$$

This incremental structure is what makes pruning possible: we can reject a bad direction before completing the whole assignment.

3.3 Explore

To explore means: choose one candidate extension for the current partial solution and descend into the corresponding branch of the search tree.

If the current node is $(x_1, \dots, x_i)$, then exploring corresponds to selecting a value for x_{i+1} .

In tree language, this means moving from a node to one of its children.

3.4 Test Early

After making a choice, the algorithm checks whether the new partial solution is still feasible.

$$C(x_1, \dots, x_i, x_{i+1})$$

If the constraints are already violated, there is no reason to continue deeper into that branch.

Key principle: detect failure as soon as possible.

3.5 Dead Ends

A branch becomes a dead end when the current partial solution cannot be extended into any valid complete solution.

This may happen because:

- a constraint is violated immediately
- no valid next choice remains

- the branch structure makes completion impossible

At that point, the algorithm stops descending and returns.

3.6 Undo

Undoing a choice means removing the most recent decision and restoring the previous search state.

Conceptually:

$$(x_1, \dots, x_i, x_{i+1}) \rightarrow (x_1, \dots, x_i)$$

This is why the technique is called backtracking: the algorithm goes back to the previous level and tries another option.

3.7 Why Recursion Fits Naturally

The search space has a tree structure, and recursion is the natural mechanism for traversing trees.

Each recursive call corresponds to:

- one node in the search tree
- one partial solution
- one active level of decision-making

Returning from recursion corresponds exactly to undoing the last choice.

3.8 The Full Intuition

The complete mechanism can be summarized as:

1. Build a partial solution
2. Try one possible next choice
3. Test whether the branch is still feasible
4. If yes, continue recursively
5. If no, undo the choice and try another one

Core mental model: backtracking is a disciplined walk through a search tree, guided by feasibility tests and reversibility of choices.

4 Generic Backtracking Algorithm

4.1 Roadmap

- The generic structure of backtracking algorithms
- The role of recursive exploration
- Choice generation and feasibility testing
- How solutions are detected
- The canonical pseudocode template

Now that we understand the intuition behind backtracking, we can formalize the technique as a generic algorithm.

Many classical problems share the same structure, even though their constraints and search spaces differ.

For this reason, backtracking is best understood through a generic algorithmic template.

4.2 State Representation

At any point in the algorithm, we maintain a partial solution.

$$(x_1, x_2, \dots, x_i)$$

This sequence represents the decisions made so far.

The state of the algorithm therefore consists of:

- the current partial assignment
- the depth in the search tree
- the remaining possible choices

4.3 Generating Choices

From a partial solution $(x_1, \dots, x_i)$, the algorithm generates possible extensions for the next decision variable.

These values typically come from a domain of feasible candidates.

Each candidate corresponds to a child node in the search tree.

4.4 Feasibility Test

Before exploring a branch further, the algorithm verifies whether the new partial solution still satisfies the constraints.

$$C(x_1, \dots, x_i, x_{i+1})$$

If the constraint check fails, the branch is pruned immediately.

Goal: eliminate impossible solutions early.

4.5 Detecting a Complete Solution

A complete solution is obtained when all decision variables have been assigned.

In other words, when the depth of the search tree reaches the required level.

At that moment, the algorithm may:

- record the solution
- print the solution
- update an optimal value
- stop the search (if only one solution is needed)

4.6 Generic Backtracking Template

The general structure of a backtracking algorithm can be written as follows:

```
BACKTRACK(state):  
  
    if state is a complete solution:  
        report solution  
        return  
  
    for each candidate choice c:  
  
        if c is feasible:  
            apply choice c  
  
            BACKTRACK(updated state)  
  
            undo choice c
```

This template captures the entire mechanism: explore a choice, recurse, and undo the choice afterward.

4.7 Structure of the Search Tree

The algorithm implicitly explores a tree structure.

- Each node represents a partial solution
- Each edge represents a decision
- Leaves correspond to complete assignments

The power of backtracking comes from the ability to prune large portions of this tree before fully exploring them.

4.8 Why This Template Is Important

Many classical algorithms are direct instantiations of this template.

- N-Queens
- Sudoku solvers
- Permutation generation

- Subset generation
- Graph coloring
- Hamiltonian path search

In each case, the differences lie in the constraint checks and candidate generation, not in the underlying search strategy.

5 Search Tree & Recursion Structure

5.1 Roadmap

- How the search tree is structured
- How recursion maps to the tree
- The relationship between depth and decisions
- How the call stack evolves during execution
- Why backtracking naturally follows DFS

Backtracking algorithms explore a conceptual structure: the search tree.

At the same time, they are implemented using recursion. Understanding how these two views align is essential.

5.2 The Search Tree

The search space of a backtracking problem can be visualized as a tree.

- The root represents the empty solution
- Each level corresponds to one decision
- Each edge represents a choice
- Each node represents a partial solution

A node at depth i corresponds to a sequence $(x_1, \dots, x_i)$.

5.3 Depth and Decision Levels

The depth of a node in the tree corresponds directly to the number of decisions made.

If a problem has n variables, the tree has at most n levels.

Key idea: depth = number of assigned variables.

5.4 Recursion as Tree Traversal

Each recursive call corresponds to one node in the search tree.

- Entering a recursive call = moving down the tree
- Returning from a call = moving back up

Therefore, the algorithm performs a depth-first traversal of the search tree.

5.5 The Call Stack

The recursion stack stores the path from the root to the current node.

At any moment, the stack contains:

- the current partial solution
- the active sequence of decisions

Stack depth = tree depth.

5.6 Why Backtracking Uses DFS

Backtracking naturally follows a depth-first strategy because it explores one branch fully before trying another.

This is efficient because:

- it uses limited memory (only one path at a time)
- it allows early pruning of invalid branches

5.7 Visual Intuition

You can think of backtracking as:

1. Going down a path (making choices)
2. Reaching a dead end or solution
3. Going back up (undo)
4. Trying a different branch

This “go down → go up → try again” pattern is exactly DFS with undo.

5.8 Why This Matters

Understanding this structure is critical because:

- it explains how recursion works internally
- it clarifies how pruning reduces complexity
- it helps reason about performance

Without this mental model, backtracking often feels like “magic recursion”.

6 Pruning

6.1 Roadmap

- Why pruning is essential
- How constraints reduce the search space
- Early vs late pruning
- Bounding and optimization
- Impact on time complexity

Backtracking without pruning is often impractical.

The search tree grows exponentially, and exploring all branches quickly becomes infeasible.

Pruning is what makes backtracking efficient.

6.2 Why Pruning Matters

In many problems, the full search tree is enormous.

However, most branches do not lead to valid solutions.

Goal: avoid exploring useless branches.

Pruning allows the algorithm to skip entire subtrees that cannot produce a valid solution.

6.3 Constraint-Based Pruning

The most basic form of pruning comes from constraints.

If a partial solution violates constraints, we stop exploring immediately.

if $\neg C(x_1, \dots, x_i) \rightarrow \text{prune}$

This is called early pruning.

6.4 Early vs Late Pruning

Pruning can happen at different stages:

- Early pruning — reject as soon as possible
- Late pruning — reject after deeper exploration

The earlier we prune, the more work we save.

6.5 Bounding (Optimization Problems)

In optimization problems, we can prune using bounds.

If a partial solution cannot beat the best known solution, it can be discarded.

if $\text{bound}(\text{state}) \leq \text{best} \rightarrow \text{prune}$

This idea is the foundation of **Branch and Bound**.

6.6 Impact on Complexity

In theory, backtracking is exponential.

But with effective pruning, the actual number of explored nodes can be dramatically smaller.

- Worst case: exponential
- With pruning: often much faster in practice

6.7 Intuition

Without pruning:

explore everything → slow

With pruning:

explore only promising paths → fast

6.8 Summary

- Pruning eliminates useless branches
- Constraints enable early pruning
- Bounds enable optimization pruning
- Efficiency depends heavily on pruning quality

Key takeaway: backtracking is only powerful when combined with pruning.

7 Classic Problems

7.1 Roadmap

- Why backtracking appears in many classical problems
- How different problems share the same search structure
- What changes from one problem to another
- How constraints define pruning strategies
- Why these examples matter in algorithms and interviews

Backtracking is not a single problem-specific trick.

It is a general search strategy that appears in many classical problems from combinatorics, graph theory, and constraint satisfaction.

The purpose of this part is to connect the generic algorithm with concrete and well-known applications.

7.2 Permutation Generation

One of the simplest applications of backtracking is the generation of all permutations of a set.

At each level, the algorithm chooses one element that has not yet been used.

- Partial solution = current permutation prefix
- Choices = unused elements
- Goal = complete permutation

This is often the first problem used to introduce backtracking.

7.3 Subset Generation

Another classical example is generating all subsets of a set.

Each decision answers a binary question: include an element or exclude it.

- Each level corresponds to one element
- Each node branches into two possibilities
- The leaves represent complete subsets

This makes the search tree especially clear and easy to visualize.

7.4 N-Queens

The N-Queens problem is one of the most famous backtracking problems.

The task is to place n queens on an $n \times n$ chessboard so that no two queens attack each other.

- Partial solution = queens placed so far
- Choices = possible columns for the next row
- Constraints = no shared column or diagonal

N-Queens is a canonical example because pruning makes a dramatic difference.

7.5 Sudoku

Sudoku can also be solved with backtracking.

The algorithm fills empty cells one by one, testing whether each placement respects the puzzle rules.

- Choices = digits that can be placed in a cell
- Constraints = row, column, and subgrid consistency
- Goal = fill the entire board

This is a strong example of constraint-driven pruning.

7.6 Graph Coloring

In graph coloring, we assign colors to vertices so that adjacent vertices receive different colors.

A backtracking solution assigns colors one vertex at a time.

- Partial solution = colors assigned so far
- Choices = available colors
- Constraint = adjacent vertices cannot share a color

This example connects backtracking directly to graph theory.

7.7 Hamiltonian Path

The Hamiltonian Path problem asks whether a graph contains a path that visits every vertex exactly once.

Backtracking explores possible vertex sequences while rejecting branches that repeat vertices or break adjacency.

- Partial solution = current path
- Choices = admissible next vertices
- Goal = a path containing all vertices

This is a classical example where the search can become very expensive.

7.8 The Common Structure

Although these problems look different, they all share the same abstract structure:

- build a partial solution incrementally
- generate possible next choices
- test feasibility
- recurse

- undo and try another branch

What changes from one problem to another is not the search strategy, but the definition of state, choices, and constraints.

8 Complexity

8.1 Roadmap

- Why backtracking is usually exponential
- The role of branching factor and depth
- Worst-case time complexity
- Space complexity and recursion depth
- Why pruning changes practice more than theory

Backtracking is powerful, but it is also expensive.

Its running time depends on how many nodes of the search tree are actually explored.

In the worst case, this number grows exponentially.

8.2 Complexity Comes from the Search Tree

The complexity of backtracking is determined by the size of the search tree.

Each node represents a partial solution, and the algorithm may need to visit many such nodes.

$$\text{Time complexity} \approx \text{number of explored nodes}$$

8.3 Branching Factor

Let b be the branching factor, that is, the maximum number of children a node may have.

If each decision offers many choices, the tree expands very quickly.

$$b = \text{number of possible choices per level}$$

Larger branching factor usually means larger running time.

8.4 Depth of the Tree

Let d be the maximum depth of the search tree.

In many problems, depth corresponds to the number of decision variables.

$$d = \text{number of decisions required for a complete solution}$$

The deeper the tree, the more possible paths the algorithm may need to explore.

8.5 Worst-Case Time Complexity

If each node has up to b children and the tree has depth d , then the number of nodes may be on the order of:

$$O(b^d)$$

This is the standard worst-case complexity of backtracking.

Important: this is why backtracking is often exponential.

8.6 Examples

Different problems lead to different values of b and d .

- Subset generation: $b = 2, d = n \rightarrow O(2^n)$
- Permutation generation: roughly $O(n!)$
- N-Queens: exponential in the worst case
- Hamiltonian Path: exponential in the worst case

So even though the generic template is the same, the concrete cost depends strongly on the problem.

8.7 Space Complexity

Backtracking usually uses much less memory than breadth-first methods.

Since it explores one path at a time, the main memory cost comes from the recursion stack and the current partial solution.

$$Space = O(d)$$

More precisely, space is proportional to the maximum recursion depth, plus the state representation.

8.8 The Effect of Pruning

In theory, pruning does not change the worst-case class: backtracking often remains exponential.

But in practice, pruning can reduce the number of explored nodes dramatically.

- Worst-case complexity may stay exponential
- Observed runtime may improve enormously

Key distinction: pruning often changes practical performance more than asymptotic theory.

8.9 Main Takeaway

Backtracking is expensive because it explores a combinatorial search tree.

Its cost is mainly controlled by:

- branching factor
- depth
- quality of pruning

This is why problem modeling and constraint design are just as important as the recursive code itself.

9 Backtracking vs DFS vs Dynamic Programming

9.1 Roadmap

- Why these techniques are often confused
- How backtracking differs from plain DFS
- How backtracking differs from Dynamic Programming
- What kind of problems each method solves best
- How to choose the right tool in practice

Backtracking, Depth-First Search (DFS), and Dynamic Programming (DP) are all fundamental techniques in algorithm design.

They sometimes look similar because all three may involve recursion, state transitions, and structured exploration.

However, they solve different kinds of problems and rely on different core ideas.

9.2 Backtracking vs DFS

Backtracking is often implemented as a depth-first traversal of a search tree.

That is why it may look like DFS.

- **DFS:** traversal strategy for graphs or trees
- **Backtracking:** search method for constructing solutions under constraints

Main distinction: DFS tells us *how to explore*; backtracking tells us *what to explore and when to undo*.

9.3 DFS as a Traversal Mechanism

DFS is a generic traversal pattern: go deep first, then return.

It is used in:

- graph traversal
- cycle detection
- topological sorting
- connected components

DFS does not, by itself, define constraints, candidate choices, or solution construction rules.

9.4 Backtracking as Constrained Search

Backtracking uses a depth-first pattern, but its goal is different.

It incrementally builds a candidate solution, checks feasibility, and undoes choices when necessary.

- build partial solution
- test constraints

- prune invalid branches
- undo and try another option

So backtracking can be seen as DFS + constraints + undo logic.

9.5 Backtracking vs Dynamic Programming

Backtracking and Dynamic Programming are often compared because both may solve combinatorial problems.

But they are based on very different ideas:

- **Backtracking:** explore possibilities and prune
- **Dynamic Programming:** reuse solutions to overlapping subproblems

Main distinction: backtracking searches; DP stores and reuses computed results.

9.6 Overlapping Subproblems

Dynamic Programming becomes powerful when the same subproblem appears many times.

In that case, recomputing it is wasteful.

same state → same answer

DP avoids repeated work by memoization or tabulation.

9.7 When DP Replaces Backtracking

Some problems that seem to invite backtracking can actually be solved more efficiently with DP.

This happens when:

- subproblems overlap heavily
- optimal substructure exists
- the state can be represented compactly

In such cases, DP may reduce exponential recursion to polynomial time.

9.8 Practical Comparison

Technique	Main Idea	Best For
DFS	Traverse deeply first	Graph and tree exploration
Backtracking	Search with constraints and undo	Constraint satisfaction and combinatorial search
Dynamic Programming	Reuse repeated subproblems	Optimization and overlapping subproblems

9.9 How to Choose

A useful practical rule is:

- Use **DFS** when the problem is mainly traversal
- Use **Backtracking** when you must build solutions under constraints
- Use **Dynamic Programming** when repeated subproblems can be reused

Important: some problems combine these ideas. For example, a recursive backtracking solution may later be optimized with memoization.

9.10 Series Conclusion

Backtracking is best understood not as “just recursion,” but as a disciplined method for exploring a constrained search space.

Across this series, we studied its problem model, recursive mechanism, search tree structure, pruning, classical applications, complexity, and relationship to other major techniques.

Source Pages

- [Overview](#)
- [Part 1](#)
- [Part 2](#)
- [Part 3](#)
- [Part 4](#)
- [Part 5](#)
- [Part 6](#)
- [Part 7](#)
- [Part 8](#)